MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Single-GPU mixture-of-experts (MoE) inference can stall on HDD reads when expert weights exceed the HBM and host memory assigned to a task. Managing HBM and host residency independently can discard experts that later tokens reuse. We present MECC (MoE Expert Cache Compiler), which couples profile-guided initial placement with joint HBM/CPU retention and dependency-aware weight delivery. The compiler assigns per-layer capacities; the runtime updates resident experts from unchanged router selections. In single-run FP16 Mixtral-8x7B measurements on one A100, MECC with uniform initialization delivers decode speedups of 1.77× over local streaming and 2.95× over fixed-CPU retention, with matching sample perplexity.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models increase parameter capacity while activating only a small subset of experts for each token [1]–[4]. This sparsity reduces computation, but inference must still make every expert addressable. A single GPU often cannot retain the full expert set. The host allocation of an individual serving task can also be smaller than the server's physical RAM. When these fast tiers cannot hold the expert working set, selected weights must be read from HDD and copied to the GPU before execution. The resulting delay recurs across layers and generated tokens. Sparse arithmetic alone therefore does not provide low-latency deployment. The problem addressed in this paper is how to use bounded HBM and host capacity to reduce expert delivery on the critical path of single-GPU inference, while preserving the model's routing and FP16 weights. This setting arises when a service reserves one accelerator and a bounded host allocation for a model, or when several jobs share the same server. The backing store provides capacity, but every demand read can delay the next layer. Deployment must therefore exploit reuse across generated tokens, rather than repeatedly paying the full delivery cost for each sparse selection.

Existing systems exploit several forms of sparsity and concurrency. Expert parallelism distributes weights over multiple GPUs [5]–[7]. Offloading systems use activation traces, CPU execution, proactive caching, and transfer pipelines [8]–[12]. SSD-LLaMA and edge FlashMoE further study disk delivery and expert replacement [13], [14]. These approaches make the choice of resident experts central to performance. Three decisions interact: which experts occupy the initial cache, what happens to an HBM victim, and how residency changes as later requests arrive. An expert displaced from HBM may soon be reused, and an initial profile may stop representing the active workload. Figure 1 isolates the cross-tier coordination problem with local A100 controls. Independently managed caches take 32.94 s per decode token, compared with 7.38 s under joint retention at the same nominal slots and initialization. Their reported HDD-demand shares are 47.9% and 12.5%. The comparison motivates treating placement and later retention as connected decisions. Choosing a good initial set alone does not determine which experts survive later GPU evictions. An online replacement rule alone also cannot recover the initial tokens already spent loading a poorly chosen set. The design question is how to carry useful information across these two time scales while keeping their responsibilities explicit.

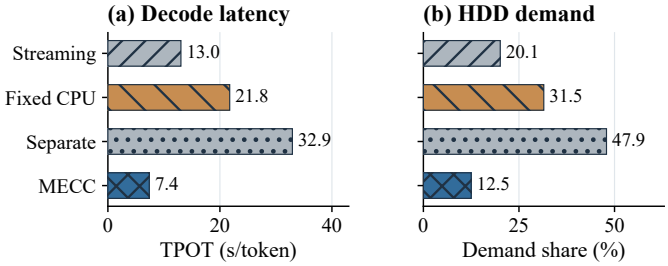


Fig. 1. Local single-A100 cache controls on Mixtral-8x7B. Separate denotes independent HBM/CPU management. All policies use the same nominal slots, initialization, and eight-prompt input. HDD demand spans all instrumented phases; each policy has one run.

We present MECC, the MoE Expert Cache Compiler, to connect a profile-derived initial state with online expert residency. The compiler ranks experts within each layer and distributes HBM and CPU slots according to the additional demand that each slot covers. It emits both capacities and the corresponding layer/expert identities. The runtime then consumes the unchanged router's selections. A CPU hit promotes its expert to HBM and retains the displaced HBM expert in CPU memory, preserving reuse across the tier boundary. Recency updates the resident identities as the input stream changes, while compiled capacities remain fixed. A dependency-aware delivery path stages missing weights and orders their copies before GPU consumption. The three parts operate at different time scales: profiling determines an initial prior, joint retention follows actual demand, and delivery schedules the weights required by the current gate. This organization preserves a single identity for each layer/expert pair from the profile to execution. A placement decision becomes an initial runtime state, and each later request updates that state through explicit promotion and demotion. The runtime can therefore respond to changed demand without rebuilding the plan or altering the model.

This paper makes three contributions.

(1) **Placement compilation:** we define a per-layer plan that retains expert identity and allocates bounded fast-tier slots by marginal covered demand.

(2) **Joint retention and delivery:** we design coordinated promotion and victim demotion, together with explicit readiness and buffer-lifetime rules for the selected experts.

(3) **Evaluation:** we implement a single-GPU harness and characterize runtime latency, source-tier demand, capacity sensitivity, and numerical behavior. On one A100 with uniform initialization, MECC takes 7.38 s/token versus 13.04 for local streaming and 21.75 for fixed CPU retention. The evaluation further organizes model, mechanism, startup, and deployment comparisons around the decisions made by the compiler and runtime. The component comparisons separate the benefit of covering early demand from the benefit of retaining experts across later evictions. Resource sweeps then connect these mechanisms to the capacity available to the serving task.

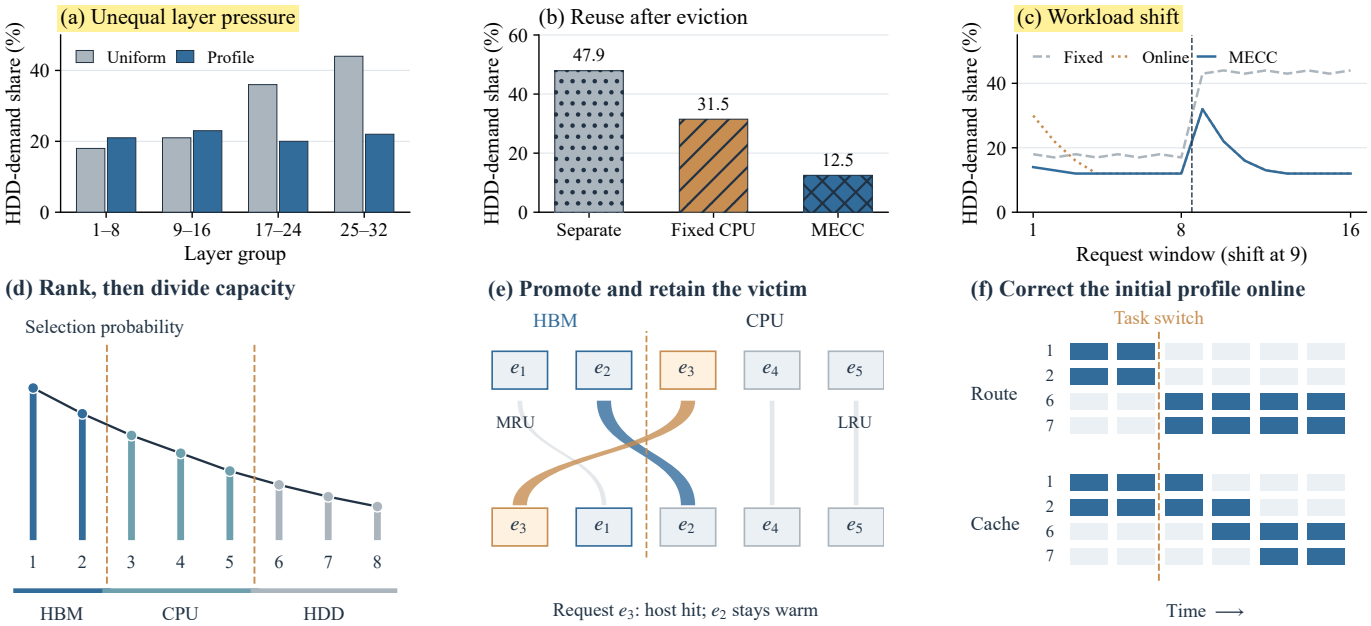


Fig. 2. Motivation evidence and matching insights. (a–c) compare layer pressure, cross-tier reuse, and workload change; (b) counts A100 demand across all phases. (d–f) show layer-specific capacity cuts, joint victim retention, and online membership updates.

II. Background

A. Sparse Execution, Full Weight Capacity

An MoE layer contains a router and a bank of experts. The router selects a small subset for each token, and their weighted outputs form the layer result [1], [2], [15]. Mixtral and the DeepSeek family use this separation between active arithmetic and total model capacity [3], [4], [16]–[18]. Expert identity includes the layer: equal indices in different layers refer to different weights. A generated token passes through successive layers, so expert demand is revealed incrementally during the forward pass. Autoregressive decoding revisits these layers with potentially different selections. Residency changes the location of an immutable expert, while its router score and function remain unchanged.

B. Expert Residency on One GPU

With enough aggregate HBM, expert parallelism places experts on different GPUs and moves token activations to their owners [5]–[7], [19], [20]. A single-GPU deployment instead moves selected weights through the memory hierarchy. Dense layers, activations, and the KV cache compete with experts for HBM; KV management is complementary to expert caching [21], [22]. A CPU hit needs an H2D copy, while an HDD miss first reads the expert into host memory. Heterogeneous inference systems expose these capacity and delivery choices [23]–[27]. We use immutable FP16 experts and retain backing weights on HDD. A resident copy avoids a later cold read, making the history of expert reuse relevant even though routing remains sparse. Below full fast-tier coverage, requests can miss both HBM and CPU. MECC addresses this regime by preserving useful copies that avoid another HDD read.

III. Motivation and Insights

A. Initial Demand Is Uneven across Layers

Equal expert counts do not imply equal cache pressure. Layers can differ in how much demand is concentrated in their most frequent experts, and routing locality also varies across models [28]. Figure 2(a,d) connects this imbalance to initial placement. A uniform allocation spends the same number of slots on layers whose next expert may cover very different request mass. The insight is to rank experts within each layer and compare the next uncovered rank across layers. Compilation turns the profile into a capacity allocation and a concrete set of initial expert IDs. The next slot can cover little demand in a layer whose hot set already fits. Comparing marginal demand captures this diminishing return under an unchanged total allowance.

B. Reuse Continues beyond an HBM Eviction

An HBM eviction changes the appropriate storage tier, not the expert's potential usefulness. If the evicted weights remain on the host, a later selection can avoid HDD. Figure 2(b,e) illustrates the coordination required: promoting one CPU expert creates room for the HBM victim in the same logical fast-tier set. In the matched local control, independent caching reports 47.9% HDD demand, versus 12.5% for joint retention across the instrumented phases. The insight is to manage the two fast tiers as a shared reuse order, so GPU turnover does not discard host-level reuse. Promotion frees a CPU position at the moment the GPU victim loses its HBM position. Reusing that host position preserves a recent expert instead of an older CPU resident.

C. A Profile Should Initialize, Not Freeze, Residency

A profile summarizes a workload; it does not specify the order of future selections. A new prompt can change the active experts before a fixed resident set adapts. Figure 2(c,f) pairs this change in demand with the delayed change in residency. The insight is to preserve the compiler's capacities while updating the resident identities online. The prior provides useful initial coverage, and observed recency corrects it as the stream evolves. This division connects deployment-time planning with inference-time behavior without changing the router or predicting an unseen gate. This also separates two effects that a fixed-profile comparison can conflate. A useful profile reduces early misses, whereas online replacement determines recovery after the demand distribution changes. An effective initialization need not remain the best resident set throughout the run. The compiler supplies a starting point; the request stream supplies the subsequent corrections.

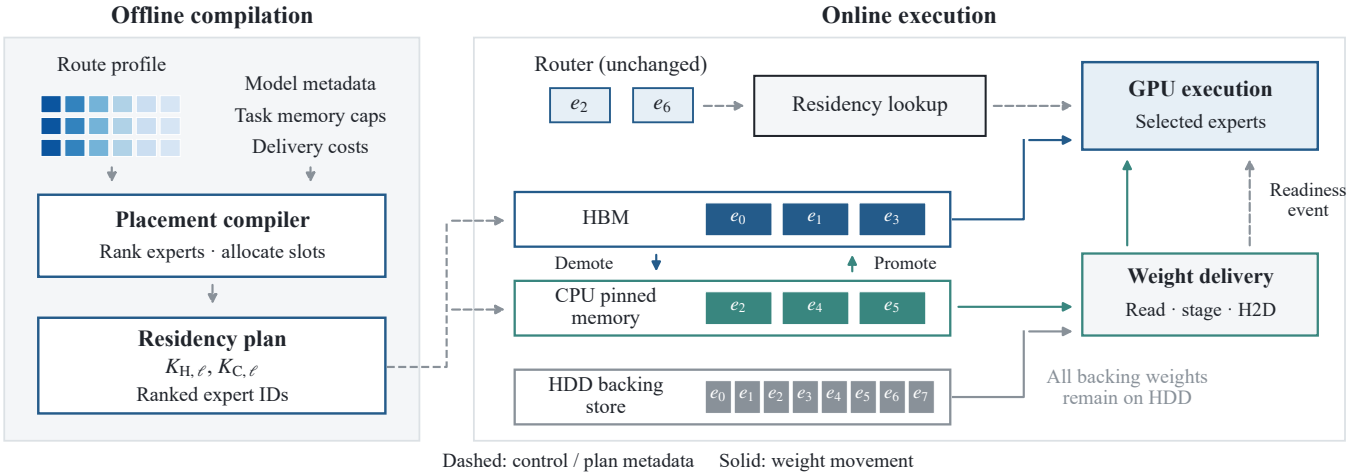


Fig. 3. MECC framework. The placement compiler emits per-layer capacities and ranked IDs. Online execution resolves unchanged router selections against HBM, CPU pinned memory, and HDD, then delivers weights before their GPU consumers. Dashed arrows carry control or plan metadata; solid arrows move weights.

Figure 3 organizes MECC around one plan and two online decisions. Offline compilation takes a route profile, model metadata, and the task's fast-tier allowances. It returns per-layer capacities and ranked expert IDs. Initialization fills HBM and CPU from those IDs; HDD remains the backing store for every expert.

During inference, each gate supplies its selected IDs to the residency lookup. An HBM hit is ready for execution. A CPU hit promotes its expert, and an HDD miss stages a new copy. The runtime preserves an HBM victim on the host when it updates the two resident sets. Weight delivery then records readiness before the GPU consumes a copied expert. This path repeats at each MoE layer and for each generated token.

The compiler fixes how much capacity each layer receives. The runtime chooses which experts occupy it as requests arrive. The delivery path orders reads and copies before each selected expert executes. Sections V and VI follow these decisions in the same order. The interfaces make each decision independently testable: a placement control changes the emitted plan, a retention control changes the membership transition, and a delivery control changes scheduling for the same selected weights. All three consume the same model metadata and preserve the original expert namespace.

V. Method

A. Compiling Capacities and Initial Identities

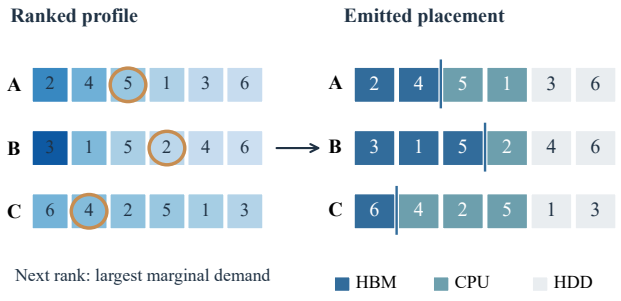


Fig. 4. Compiling the initial state. The left grid contains ranked per-layer demand; circles mark the next uncovered candidates. Shared HBM/CPU slot allowances produce different cuts in the emitted rows. A, B, and C identify different layers. Values and slot counts are schematic.

Consider L MoE layers with N_ℓ experts per layer. The evaluated models use equal-size experts and equal top-k across layers. Let s be expert size and let B_H, B_C be logical expert allowances. They supply $S_H = \lfloor B_H/s \rfloor$ HBM slots and $S_C = \lfloor B_C/s \rfloor$ CPU slots. Dense weights, activations, mirrors, and staging are separate allocations.

The compiler takes an expert order and a skew parameter α_ℓ for each layer. Profile counts determine expert order; a discrete Zipf model supplies the marginal slot score:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{\sum_{j=1}^{N_\ell} j^{-\alpha_\ell}}, \qquad p_{\ell,r} = q_{\ell,r}/L. \tag{1}$$

Here r is a rank within one layer, not an expert ID shared across layers. Equal top-k gives equal layer selection mass. Sorting counts preserves the mapping from each rank to its original expert ID.

Figure 4 shows the resulting cuts. A valid plan obeys $\sum_\ell K_{H,\ell} \le S_H$, $\sum_\ell K_{C,\ell} \le S_C$, and $K_{H,\ell} + K_{C,\ell} \le N_\ell$. The adaptive scan starts with one slot per layer in each fast tier. We consider allowances that support these minima and leave CPU ranks after the HBM cut.

The scan first assigns HBM slots. At each step, it chooses the layer with the largest next-rank score:

$$\Delta_\ell = q_{\ell,K_{H,\ell}+1}. \tag{2}$$

CPU allocation then compares ranks after each HBM cut. Allocating a slot exposes the next rank in that layer. Layer ID breaks score ties; expert ID breaks count ties. A direct scan costs $O(L(S_H + S_C))$ after sorting. It maximizes marginal coverage at each step; it does not solve a joint latency objective.

The plan emits per-layer capacities and ordered IDs. Initialization places the first HBM ranks on the GPU and the next CPU ranks on the host. Missing count data selects natural ID order. Capacities stay fixed during a run. Router requests can replace initial residents, including experts absent from the profile.

B. Retaining Victims across the Tier Boundary

Each layer keeps two ordered resident sets. Most-recently used (MRU) entries precede least-recently used (LRU) entries. HBM holds the first part of the logical order; CPU holds the next part. An HBM hit moves its expert to HBM MRU. A CPU hit removes its host entry and promotes it to HBM MRU. If HBM is full, its LRU victim

becomes CPU MRU. CPU pressure removes the CPU LRU. HDD retains every backing expert.

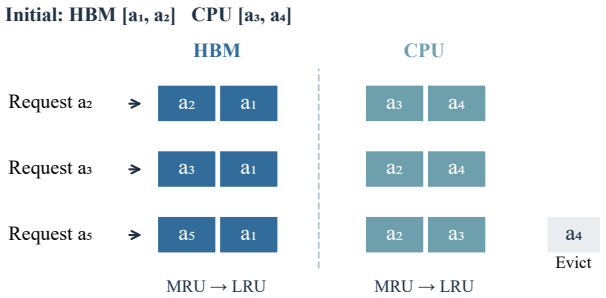


Fig. 5. Three requests from the same initial state. A hit changes recency; a host promotion also demotes the GPU victim; a cold admission can additionally evict the CPU victim. Rows show completed logical states, each ordered MRU to LRU.

Figure 5 starts each row from HBM $[a_1, a_2]$ and CPU $[a_3, a_4]$. Requesting a_2 changes only HBM recency. Requesting a_3 gives HBM $[a_3, a_1]$ and CPU $[a_2, a_4]$. Requesting cold a_5 gives HBM $[a_5, a_1]$ and CPU $[a_2, a_3]$; a_4 leaves the fast tiers. Promotion preserves the more recent GPU victim in the position vacated on the host.

These completed serial transitions implement exclusive LRU across the combined logical capacity. Batch preparation can promote several experts before their consumers run. The serial stack describes replacement, while the batch path also needs temporary copies and a consumption order. Neither this stack rule nor an initial profile guarantees lower latency on every request stream.

The implementation retains a host mirror with each GPU resident. Demotion can reuse that mirror without D2H. Mirrors add physical host bytes; they do not add logical CPU slots. Logical eviction can release a cache entry before CUDA finishes using its storage. Tensor lifetime must therefore be tracked separately from membership.

C. Ordering Delivery before Expert Consumption

Prefill identifies the experts selected across its token rows. The runtime submits cold reads for those experts and resolves each read at consumption. Figure 6 shows a prefill schedule in which a resident expert executes while another read progresses. Decode uses demand delivery rather than this prefill read batch. Both paths consume the original gate output and preserve expert aggregation order.

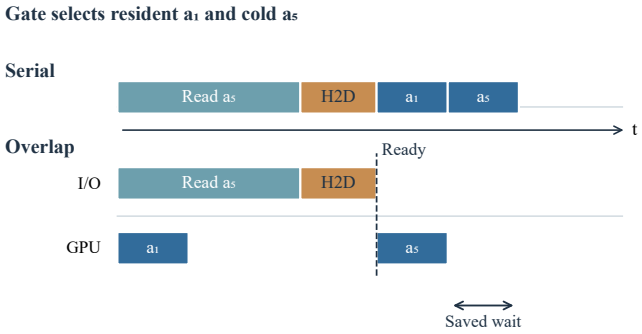


Fig. 6. Prefill delivery of resident a_1 and cold a_5 selected by one gate. Overlap executes ready work during the same read/copy path. The readiness event orders a_5's consumer; the host source remains live through copy completion. Durations are schematic.

An SSD future must complete before its host weights can be copied. The implementation then waits on the copy stream before yielding CUDA weights to a consumer. Recording the consuming stream prevents allocator reuse while kernels still reference the tensors. These operations enforce read, copy, and use dependencies. The current synchronization covers the copy stream; it is not a per-expert readiness-event scheduler.

The schedule can reduce exposed delivery time without changing which requests miss. Its overlap depends on available resident work and on the submitted read batch. Cache coverage and delivery overlap therefore require separate comparisons.

D. Sharing Execution and Accounting

Policies share the FP16 loader and router. Accounting distinguishes unique primary residents, host mirrors, and staging storage. Initialization, warmup, prefill, decode, and PPL use separate windows. Logical slot counts describe the replacement experiment; physical allocations describe the host and GPU resources it consumes.

VI. Evaluation

A. Experimental Setup

Measured runs use FP16 Mixtral-8x7B-Instruct-v0.1 on one A100 80 GB PCIe. The cold store is on a RAID logical device reporting rotational media; physical drives remain uninventoried. Main runs use four HBM and three host slots per layer, uniform initialization, eight WikiText-2 prompts, 64 input and 64 output tokens, and one run per policy. Perplexity uses 1024 WikiText-2 tokens. The 50/40 GB planner allowances define logical slots; physical allocations also include mirrors and staging buffers.

Yellow-highlighted values and panel titles denote authored illustrative targets, not measurements. The other numerical results are measured A100 outcomes. The experiments address full-system benefit, individual mechanisms, workload change, resource tradeoffs, and numerical equivalence. Streaming loads selected experts on demand without MECC's retention rules. Fixed CPU reserves a static host set; independent caching updates the two fast tiers separately. RF and CPU-compute are local adapters within the shared harness. These controls compare residency and execution decisions under one loader, rather than importing latency numbers from different machines.

B. Single-GPU Performance

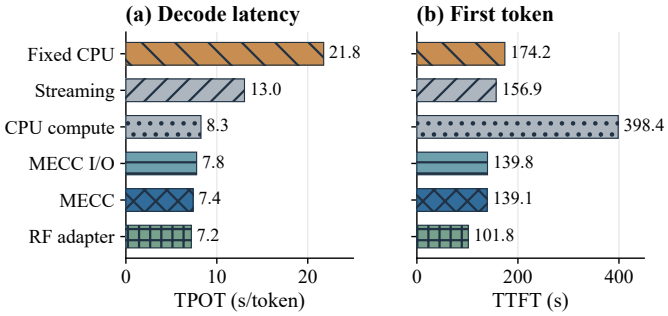


Fig. 7. Measured single-A100 runtime: one run per local policy, eight WikiText-2 prompts, 64 output tokens. RF and CPU-compute adapters are local mechanisms.

Figure 7 reports MECC at 7.38 s/token, streaming at 13.04, and fixed CPU at 21.75: TPOT ratios of 1.77× and 2.95×. The RF adapter takes 7.19 s, MECC I/O 7.78, and CPU compute 8.26; the RF adapter is the fastest local variant. MECC's TTFT is 139.1 s, versus 156.9 for streaming and 174.2 for fixed CPU. MECC, streaming, fixed CPU, RF, and MECC I/O report PPL 3.427; CPU compute reports 3.432. The short-window route/logit/token diagnostic agrees between the local streaming and MECC paths.

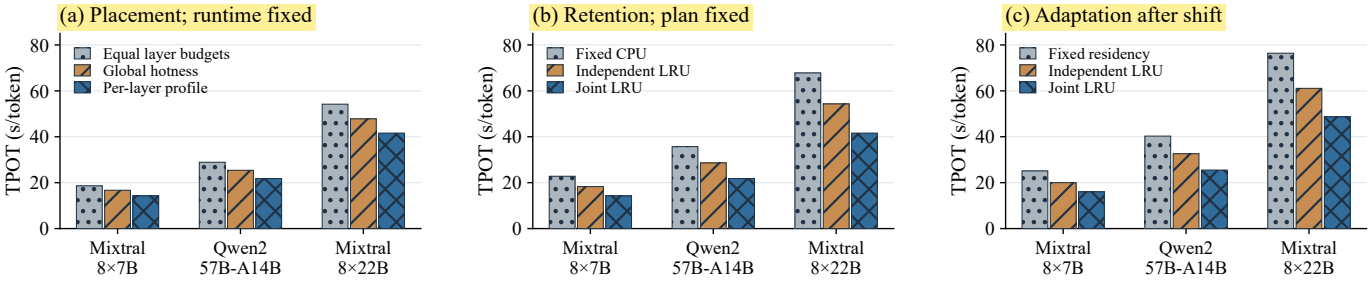


Fig. 8. Component comparisons on three models. (a) Placement with runtime fixed. (b) Retention with initialization fixed. (c) Adaptation after a task switch. The controls isolate initialization, victim retention, and online adaptation, respectively.

C. Mechanism Attribution

The matched local independent-cache control takes 32.94 s/token. Figure 8 separates three mechanisms in the target component harness. For Mixtral-8x7B, the placement target moves from 18.6 to 14.2 s/token, joint retention from 18.3 to 14.2, and post-switch adaptation from 25.1 to 16.0 relative to fixed residency. The three controls hold runtime, initial plan, and switch protocol fixed, respectively. This separates better initial coverage from preservation of later reuse.

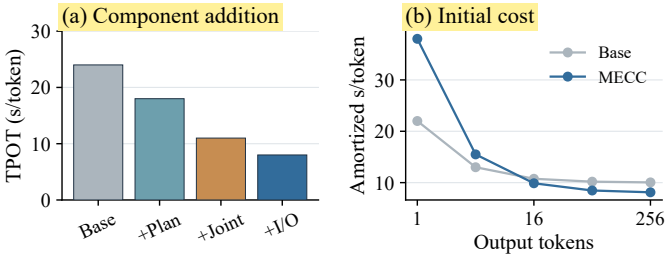


Fig. 9. Ablation and startup. The right panel reports initialization cost per output token plus decode cost.

Figure 9 tests component addition and startup amortization. The authored TPOT sequence is 24, 18, 11, and 8 s/token. The last I/O step retains a 20-percent target HDD-demand share, separating delivery scheduling from miss reduction. With the plotted startup/decode costs, the target amortization curves cross at 9 output tokens. This check exposes the service horizon over which an initial investment pays back.

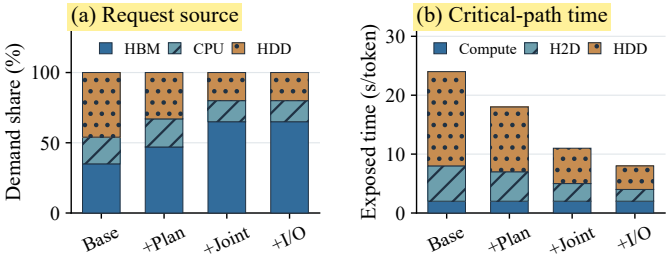


Fig. 10. Source demand and exposed time. The last I/O step preserves the demand fractions while shortening the exposed read/copy path.

Figure 10 separates cache coverage from scheduling. In the target decomposition, the HDD share falls from 46 to 20 percent before the I/O step. Overlap then shortens exposed delivery without changing the resident-set result. The stacked latency components sum to the TPOT targets in Figure 9. A source-demand share counts expert requests, while an exposed-time component counts delay on the execution path. Their changes need not be proportional: a host hit still requires H2D, and a cold read may overlap ready GPU work. Keeping both views distinguishes an improvement in cache coverage from an improvement in delivery timing.

D. Model and Deployment Scaling

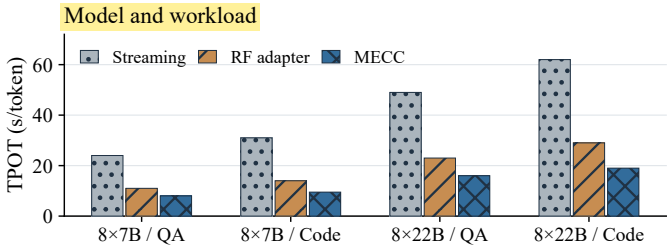


Fig. 11. Model and workload comparison. Mixtral-8x7B and 8x22B with QA/code inputs. Each policy uses the same model and workload within a group. The RF policy is a local adapter.

Figure 11 expands the present one-model workload. The MECC target ranges from 8 to 19 s/token, versus 24 to 62 for streaming and 11 to 29 for the RF adapter. QA and code exercise different expert-demand patterns; increasing model size also changes the expert pool and transfer size. The comparison therefore pairs every policy within each model/workload point.

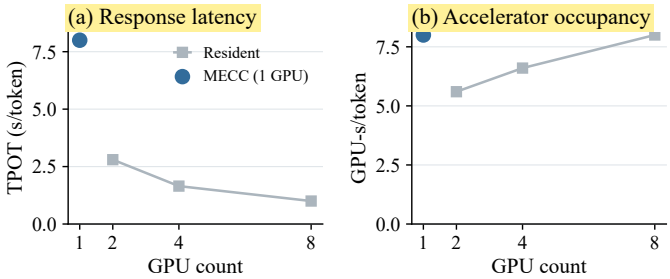


Fig. 12. One-GPU/resident-reference tradeoff. MECC has only a one-GPU point. Resident points use 2/4/8 GPUs; occupancy is GPU count times TPOT.

Figure 12 separates GPU provisioning from response latency. The endpoint target is one GPU at 8 s/token versus eight GPUs at 1 s/token; both correspond to 8 GPU-s/token. The two-GPU resident target is 5.6 GPU-s/token, which makes the smallest feasible resident configuration a necessary comparison. Occupancy is reported alongside latency, rather than substituted for energy.

E. Capacity and Workload Sensitivity

The measured 30/60 GB settings give MECC TPOTs of 14.46 and 0.76 s/token, with 32.2% and zero HDD demand. Four HBM plus four host slots at 60 GB cover all eight experts per layer; zero cold demand is a capacity transition. RF takes 13.06 and 0.74 s/token. These points use a shorter protocol than the main run.

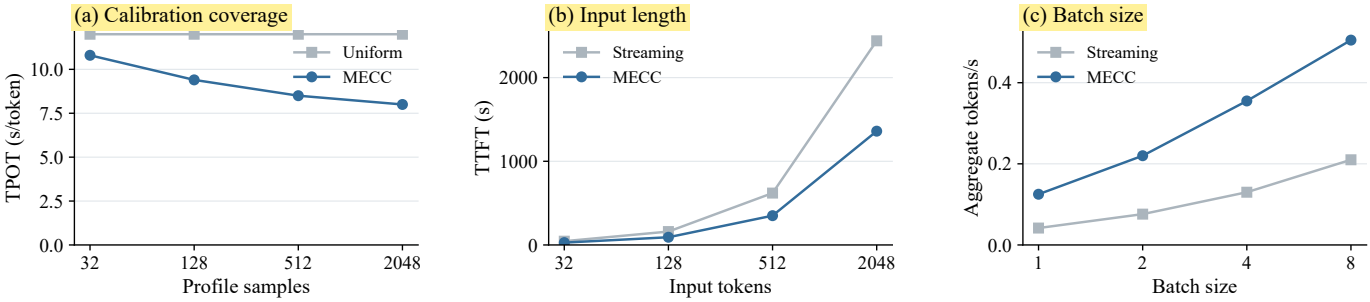


Fig. 13. Profile, input-length, and batch sensitivity. Profile size changes initialization quality, input length changes first-token work, and batch size changes aggregate throughput. The panels vary one workload parameter at a time.

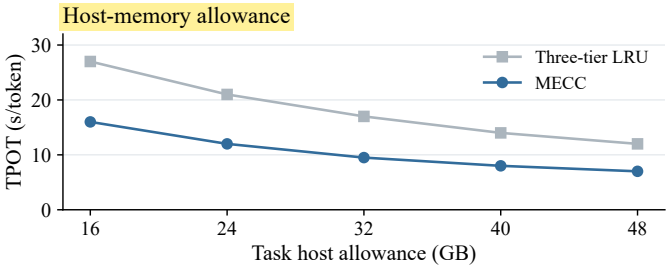


Fig. 14. Host-memory allowance sweep. Five host allowances compare MECC and three-tier LRU within one comparison protocol.

Figure 14 resolves intermediate capacity points. Its authored MECC target changes from 16 to 7 s/token over 16–48 GB, compared with 27 to 12 for three-tier LRU. The purpose is to separate pressure within the cold-tier regime from the full-coverage threshold. Additional host capacity can first preserve more reused victims, then eliminate cold demand once the complete expert set fits. The latter transition changes the dominant delivery path from HDD plus H2D to H2D alone. It explains why capacity and replacement policy must be examined together instead of extrapolating one speedup across all allowances.

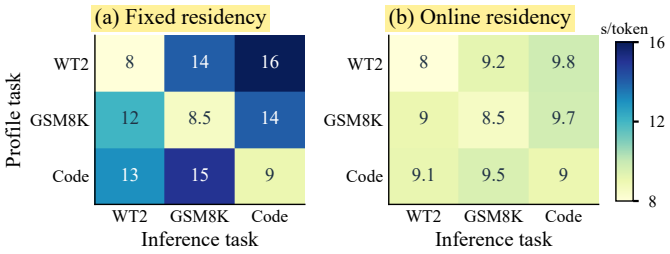


Fig. 15. Profile transfer across tasks. Rows select the calibration task and columns the inference task. Cells are TPOT in s/token; the two panels share a color scale.

Figure 13 examines scale within a workload. Figure 15 examines transfer between WikiText-2, GSM8K, and code inputs. Its target pattern distinguishes a useful initial prior from a permanently fixed placement: online membership updates reduce the off-diagonal penalty while retaining the matched-profile behavior.

VII. Related Work

Expert offloading starts from the distinction between sparse execution and full weight capacity [8], [29]. Fiddler and DAOP choose CPU/GPU execution or movement; MoE-Lightning pipelines heterogeneous work; MoE-Infinity uses activation traces [9]–[11], [30]. ProMoE, Pre-gated MoE, Fate, and ExpertFlow provide lookahead, proactive caching, or token scheduling [12], [31]–[33]. MECC's demand path instead consumes the original gate output and coordinates initial placement with online residency. The distinction concerns when information becomes available. A trace supplies a distribution before execution; a gate supplies the current demand during execution. MECC uses the former to initialize capacities and identities, and the latter to update membership. This lets the replacement mechanism remain responsive when the runtime stream differs from the calibration input.

HOBBIT and Specter use precision in expert loading; MoEpic divides experts into segments; SliceMoE caches precision slices; Mira combines staging with quantization [34]–[38]. ZipMoE uses compression and affinity scheduling [39]. SSD-LLaMA, edge FlashMoE, and MoE-CORE address storage delivery, replacement, or coordinated residency [13], [14], [40]. MECC studies the interaction between an independent placement prior and online whole-expert retention across the two fast tiers under HDD-backed demand. This fixes the cached object as an unchanged FP16 expert. Segmenting an object or storing another precision can reduce the bytes required by one miss; retaining the whole object changes whether that miss occurs. The decisions can be complementary, but their performance effects arise from different mechanisms. We therefore keep representation and routing fixed when attributing the local retention benefit.

Klotski and Bevy expose independent work across batches or requests [41], [42]. OD-MoE distributes on-demand loading over edge nodes, while HeteGen studies heterogeneous dense-model inference [43], [44]. Their execution units differ from batch-one per-gate expert delivery. The full-system, mechanism, and resource experiments above distinguish these comparisons. Cross-request schedulers can run another request while one waits for weights. The current-gate delivery path instead overlaps ready expert work from the ongoing request. Both expose concurrency, but the available work and readiness dependencies differ. Separating these execution scopes also separates their throughput and response-time effects.

VIII. Conclusion

MECC connects profile-guided placement, online cross-tier retention, and expert delivery for HDD-backed single-GPU MoE inference. Per-layer compilation improves the initial use of bounded capacity. Joint retention preserves reuse after GPU eviction, and readiness-aware delivery overlaps available work with weight movement. The A100 implementation characterizes these decisions through runtime latency, source-tier demand, capacity sensitivity, and numerical comparisons under a shared FP16 execution harness.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[4] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[5] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[6] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[7] O. J. Aimuyo, B. Oh, and R. Singh, "FlashMoE: Fast distributed MoE in a single kernel," *arXiv preprint arXiv:2506.04667v3*, 2025.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[9] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[10] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[11] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[12] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[13] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[14] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[15] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[16] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[17] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[18] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[19] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[20] Z. Doucet, R. Sharma, M. de Vos, R. Pires, A.-M. Kermarrec, and O. Balmau, "HarMoEny: Efficient multi-GPU inference of MoE models," *arXiv preprint arXiv:2506.12417v2*, 2025.

[21] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[22] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[23] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[24] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[25] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[26] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[27] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[28] J. Liang, S. Wang, M. Tian *et al.*, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," *arXiv preprint arXiv:2505.16056v4*, 2026.

[29] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[30] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," *arXiv preprint arXiv:2501.10375*, 2025.

[31] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[32] Z. Fang, Z. Hong, Y. Huang *et al.*, "Fate: Fast edge inference of mixture-of-experts models via cross-layer gate," *arXiv preprint arXiv:2502.12224v2*, 2025.

[33] X. He, S. Zhang, K. Tang *et al.*, "ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling," *arXiv preprint arXiv:2410.17954v2*, 2026.

[34] P. Tang, J. Liu, X. Hou *et al.*, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433v2*, 2024.

[35] W. Wang, X. Hou, X. Xia *et al.*, "Specter: Fast MoE inference via quantization-enabled multi-step speculative offloading," *arXiv preprint arXiv:2511.14102v3*, 2026.

[36] J. Yan, J. Liu, H. Xu, and L. Huang, "Accelerating mixture-of-expert inference with adaptive expert split mechanism," *arXiv preprint arXiv:2509.08342*, 2025.

[37] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo, "SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference," *arXiv preprint arXiv:2512.12990*, 2025.

[38] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.

[39] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

[40] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[41] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[42] H. Yu, I. Gim, K. Lee, and L. Zhong, "Accelerating offloading-based MoE inference by interleaving concurrent requests," *arXiv preprint arXiv:2611.04794*, 2026.

[43] L. Wang, Y. Du, Y. Pan, S. C. Liew, J. Liu, and K. Chen, "OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference," *arXiv preprint arXiv:2512.03927*, 2025.

[44] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Heterogeneous parallel inference for large language models on resource-constrained devices," *arXiv preprint arXiv:2403.01164*, 2024.